

# **Unit 4.**

## **Creating ASP.NET core MVC applications**

Setting up the Environment, Controllers and Actions: Create Controllers, Create Actions and Action Results Types, Rendering HTML with Views: Razor Syntax, Understanding Tag Helpers, Models: Binding and Validations, URL Routing and features, Web API Applications: API Controllers, JSON, Dependency Injection and IOC containers

**By: Yuba Raj Devkota**  
**Net Centric Computing**  
**CSIT 6<sup>th</sup> Semester**  
**NCCS College**

# Setting up the Environment

## Steps to Create an ASP.NET Core MVC Project

### Step 1: Open Visual Studio

Open Visual Studio 2022

### Step 2: Create a New Project

Choose

Create a new project

### Step 3: Select Project Template

Choose

ASP.NET Core Web App (Model-View-Controller)

Click **Next**

### Step 4: Configure Project

Example

```
Project Name : StudentMVC
Location    : D:\Projects
Framework   : .NET 8
Authentication : None
HTTPS       : Checked
```

Click **Create**

### Step 5: Project Structure

Visual Studio automatically creates the following folders.

```
StudentMVC
|
|-- Controllers
|       HomeController.cs
|
|-- Models
|
|-- Views
|       Home
|       Shared
|
|-- wwwroot
|       CSS
|       JS
|       Images
|
|-- appsettings.json
|-- Program.cs
```

Suppose the user enters

`https://localhost:7019/Home/Index`

The request flows as

Browser



`Program.cs`



`HomeController`



`Index()`



`Index.cshtml`



Response to Browser

## Controller

A Controller

- Handles HTTP requests
- Communicates with Models
- Sends data to Views
- Returns responses

Every controller

- Ends with **Controller**
- Inherits from **Controller**

## Controllers and Actions

- *Controllers receive requests from users and decide what response should be returned.*
- *An Action is a method inside a controller that performs a specific task.*

```
using Microsoft.AspNetCore.Mvc;  
  
public class StudentController : Controller  
{  
  
}
```

# Creating a Controller

## Step 1

Open

Controllers

## Step 2

Add New Item

StudentController.cs

## Step 3

Write

```
</> C#  
  
using Microsoft.AspNetCore.Mvc;  
  
namespace StudentMVC.Controllers  
{  
    public class StudentController : Controller  
    {  
    }  
}
```

## Creating Actions

An Action is a public method inside a controller.

Example

```
</> C#  
  
public IActionResult Index()  
{  
    return View();  
}
```

```
using Microsoft.AspNetCore.Mvc;

namespace StudentMVC.Controllers
{
    public class StudentController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
    }
}
```

URL

`https://localhost:7019/Student`

or

`https://localhost:7019/Student/Index`

Example `Index.cshtml`

 HTML

`<h2>Student Home Page</h2>`

`<p>Welcome to ASP.NET Core MVC.</p>`

This loads

Views  
    Student  
        Index.cshtml

**Note:** If `Views/Student/Index.cshtml` does not exist, you will get the error:

InvalidOperationException:  
The view 'Index' was not found.

## Example 2

</> C#

```
public IActionResult About()  
{  
    return View();  
}
```

Requires

Views

Student

About.cshtml

## Example 3

</> C#

```
public IActionResult Contact()  
{  
    return View();  
}
```

Requires

Views

Student

Contact.cshtml

# Action Result Types

An Action can return different types of results depending on the requirement.

## 1. ViewResult

Returns a View.

Example

```
</> C#  
  
public IActionResult Index()  
{  
    return View();  
}
```

Output

Views/Student/Index.cshtml

Most commonly used in MVC applications.

## 2. ContentResult

Returns plain text.

Example

```
</> C#  
  
public IActionResult Message()  
{  
    return Content("Welcome to ASP.NET Core MVC");  
}
```

Output

Welcome to ASP.NET Core MVC

### 3. JsonResult

Returns JSON data.

Example

</> C#

```
public IActionResult Student()
{
    return Json(new
    {
        Id = 101,
        Name = "Ram",
        Age = 20
    });
}
```

Output

</> JSON

```
{
  "id":101,
  "name":"Ram",
  "age":20
}
```

Useful for APIs and AJAX requests.

### 4. RedirectResult

Redirects to another URL.

Example

</> C#

```
public IActionResult Google()
{
    return Redirect("https://www.google.com");
}
```

The browser is redirected to Google.



## 5. RedirectToActionResult

If `Home()` is accessed, it redirects to the `Index()` action in the same controller.

Redirects to another action within the application.

Example

</> C#

```
public IActionResult Home()
{
    return RedirectToAction("Index");
}
```

## 6. FileResult

Returns a file for download.

Example

</> C#

```
public IActionResult Download()
{
    byte[] fileBytes = System.IO.File.ReadAllBytes("wwwroot/files/Notes.pdf");

    return File(fileBytes,
                "application/pdf",
                "Notes.pdf");
}
```

The browser downloads **Notes.pdf**.

## 7. EmptyResult

Returns no content.

Example

```
</> C#  
  
public IActionResult Empty()  
{  
    return new EmptyResult();  
}
```

Output

No response body.

## 8. NotFoundResult

Returns HTTP 404.

Example

```
</> C#  
  
public IActionResult Search(int id)  
{  
    return NotFound();  
}
```

Output

404 Not Found

## 9. BadRequestResult

Returns HTTP 400.

Example

```
</> C#  
  
public IActionResult Save()  
{  
    return BadRequest();  
}
```

Output

400 Bad Request

## Summary Table

| Action Result Type     | Example                                                      | Purpose                                             |
|------------------------|--------------------------------------------------------------|-----------------------------------------------------|
| ViewResult             | <code>return View();</code>                                  | Returns a Razor view (.cshtml).                     |
| ContentResult          | <code>return Content("Hello");</code>                        | Returns plain text.                                 |
| JsonResult             | <code>return Json(student);</code>                           | Returns JSON data.                                  |
| RedirectResult         | <code>return<br/>Redirect("https://google.com")<br/>;</code> | Redirects to an external URL.                       |
| RedirectToActionResult | <code>return<br/>RedirectToAction("Index");</code>           | Redirects to another action within the application. |
| FileResult             | <code>return File(...)</code>                                | Returns a file for download.                        |
| EmptyResult            | <code>return new EmptyResult();</code>                       | Returns no content.                                 |
| NotFoundResult         | <code>return NotFound();</code>                              | Returns HTTP 404 (Not Found).                       |
| BadRequestResult       | <code>return BadRequest();</code>                            | Returns HTTP 400 (Bad Request).                     |

# Rendering HTML with Views: Razor Syntax, Understanding Tag Helpers

*A View in ASP.NET Core MVC is responsible for displaying data (HTML) to the user. Views are usually written using the Razor View Engine, which combines HTML and C# code in a single file with the extension .cshtml.*

## Razor Syntax

*Razor is the view engine used by ASP.NET Core MVC to generate dynamic HTML pages. It allows you to embed C# code inside HTML using the @ symbol.*

### Features of Razor

- Mixes HTML and C# code easily.
- Simple and readable syntax.
- Automatically encodes output to prevent XSS attacks.
- Supports loops, conditions, layouts, and reusable components.

## Basic Razor Syntax

### Example 1: Displaying Text

 HTML

```
<h2>Welcome to ASP.NET Core MVC</h2>
```

```
<p>Today's Date: @DateTime.Now.ToShortDateString()</p>
```

### Output

Welcome to ASP.NET Core MVC

Today's Date: 7/26/2026

## Example 2: Variables

 C#

```
@{  
    string name = "Ram";  
    int age = 22;  
}
```

```
<h2>Student Information</h2>
```

```
<p>Name: @name</p>
```

```
<p>Age: @age</p>
```

### Output

Student Information

Name: Ram

Age: 22

## Razor Code Block

A code block is written using:

 C#

```
@{  
    // C# code here  
}
```

### Example

 C#

```
@{  
    int a = 10;  
    int b = 20;  
    int sum = a + b;  
}
```

```
<h3>Sum = @sum</h3>
```

### Output

Sum = 30

## Conditional Statements

Example

```
</> C#  
  
@{  
    int marks = 75;  
}  
  
@if (marks >= 40)  
{  
    <h3>Pass</h3>  
}  
else  
{  
    <h3>Fail</h3>  
}
```

Output

Pass

## Loops in Razor

Example using for loop

```
</> C#  
  
<ul>  
  
@for(int i=1; i<=5; i++)  
{  
    <li>Item @i</li>  
}  
  
</ul>
```

Output

- Item 1
- Item 2
- Item 3
- Item 4
- Item 5

## Example using foreach

```
</> C#  
  
@{  
    string[] subjects =  
    {  
        "C#",  
        "Java",  
        "Python"  
    };  
  
    <ul>  
  
    @foreach(var sub in subjects)  
    {  
        <li>@sub</li>  
    }  
  
    </ul>
```

Output

- C#
- Java
- Python

## Displaying Model Data

Suppose the controller sends data.

Controller

```
</> C#  
  
public IActionResult Index()  
{  
    ViewBag.Name = "Hari";  
    return View();  
}
```

View

 HTML

```
<h2>Welcome @ViewBag.Name</h2>
```

Output

Welcome Hari

# Strongly Typed View Example

## Student Model

```
</> C#  
  
public class Student  
{  
    public string Name { get; set; }  
    public int Age { get; set; }  
}
```

## Controller

```
</> C#  
  
public IActionResult Index()  
{  
    Student s = new Student()  
    {  
        Name = "Ram",  
        Age = 21  
    };  
  
    return View(s);  
}
```

## View

```
</> C#  
  
@model Student  
  
<h2>Student Details</h2>  
  
<p>Name: @Model.Name</p>  
<p>Age: @Model.Age</p>
```

## Output

Student Details

Name: Ram

Age: 21

## Razor Symbols

| Symbol        | Purpose               |
|---------------|-----------------------|
| @             | Starts C# code        |
| @{ }          | Code block            |
| @Model        | Access model object   |
| @ViewBag      | Access ViewBag        |
| @if           | Conditional statement |
| @foreach      | Loop                  |
| @DateTime.Now | Current date/time     |



# Understanding Tag Helpers

- *Tag Helpers are server-side components in ASP.NET Core MVC that help generate HTML elements. They look like normal HTML tags but automatically generate correct HTML based on ASP.NET Core features.*
- *Instead of writing long HTML manually, Tag Helpers make code cleaner and less error-prone.*

## Common Tag Helpers

| Tag Helper             | Purpose                 |
|------------------------|-------------------------|
| asp-controller         | Specifies controller    |
| asp-action             | Specifies action        |
| asp-route              | Pass route values       |
| asp-for                | Binds model property    |
| asp-validation-for     | Validation message      |
| asp-validation-summary | Shows validation errors |
| asp-items              | Populates dropdown list |



## Example 1: Anchor Tag Helper

Instead of writing

HTML

```
<a href="/Student/Index">Students</a>
```

Use

HTML

```
<a asp-controller="Student"
    asp-action="Index">
    Students
</a>
```

Generated HTML

HTML

```
<a href="/Student/Index">
    Students
</a>
```

**Advantage:** If routing changes later, ASP.NET automatically updates the generated URL.

## Example 2: Form Tag Helper

HTML

```
<form asp-controller="Student"
      asp-action="Create"
      method="post">

    <button type="submit">
        Save
    </button>

</form>
```

Generated HTML

HTML

```
<form action="/Student/Create"
      method="post">

</form>
```

The asp-for attribute automatically creates the correct id, name and value attributes based on model property.

## Example 3: Input Tag Helper

Model

C#

```
public class Student
{
    public string Name { get; set; }
}
```

View

C#

```
@model Student

<input asp-for="Name" class="form-control" />
```

Generated HTML

HTML

```
<input type="text"
      id="Name"
      name="Name"
      value="" />
```

## Example 4: Label Tag Helper

HTML

```
<label asp-for="Name"></label>
```

Generated HTML

HTML

```
<label for="Name">Name</label>
```

## Example 5: Validation Message Tag Helper

HTML

```
<span asp-validation-for="Name"></span>
```

If validation fails:

Name is required.

## Example 6: Validation Summary

HTML

```
<div asp-validation-summary="All"></div>
```

Displays all validation errors in one place.

## Example 7: Dropdown Tag Helper

Controller

C#

```
ViewBag.Courses = new SelectList(new[]  
{  
    "BCA",  
    "BIT",  
    "BIM"  
});
```

View

Generated HTML

HTML

```
<select>  
  
<option>BCA</option>  
<option>BIT</option>  
<option>BIM</option>  
  
</select>
```

HTML

```
<select asp-for="Course"  
        asp-items="ViewBag.Courses">  
</select>
```

# Complete Student Form Example

## Model

```
</> C#  
  
public class Student  
{  
    public string Name { get; set; }  
  
    public int Age { get; set; }  
}
```

## Controller

```
</> C#  
  
public IActionResult Create()  
{  
    return View();  
}  
  
[HttpPost]  
public IActionResult Create(Student s)  
{  
    return View();  
}
```

## View (Create.cshtml)

```
</> C#  
  
@model Student  
  
<form asp-action="Create" method="post">  
  
    <div>  
        <label asp-for="Name"></label>  
  
        <input asp-for="Name" />  
  
        <span asp-validation-for="Name"></span>  
    </div>  
  
    <div>  
        <label asp-for="Age"></label>  
  
        <input asp-for="Age" />  
  
        <span asp-validation-for="Age"></span>  
    </div>  
  
    <button type="submit">  
        Save  
    </button>  
  
</form>
```

*This example demonstrates the use of several Tag Helpers together {asp-action, asp-for, and asp-validation-for} to create a clean, strongly typed form with minimal HTML.*

# Models: Model Binding and Validation

*A Model is a C# class that represents the application's data and business rules. It is used to transfer data between the Controller and the View.*

For example, a student registration form can use a `Student` model.

`</> C#`

```
public class Student
{
    public int Id { get; set; }

    public string Name { get; set; }

    public int Age { get; set; }
}
```

- *Model Binding is the process by which ASP.NET Core MVC automatically maps data from an HTTP request (such as form fields, query strings, or route values) to the parameters of an action method or to a model object.*
- *Instead of manually retrieving values from the request, ASP.NET Core performs the mapping automatically.*

## Sources of Model Binding

- Form data
- Route values
- Query string
- HTTP headers
- JSON body (Web APIs)

# How Model Binding Works

## Step 1: User fills the form

Name = Ram  
Age = 21



## Step 2: Browser sends POST request



## Step 3: ASP.NET Core creates Student object



## Step 4: Controller receives populated object

## Example

### Model

</> C#

```
public class Student
{
    public string Name { get; set; }

    public int Age { get; set; }
}
```

## Controller

</> C#

```
public class StudentController : Controller
{
    public IActionResult Create()
    {
        return View();
    }

    [HttpPost]
    public IActionResult Create(Student student)
    {
        return Content($"Name: {student.Name}, Age: {student.Age}");
    }
}
```

## View (Create.cshtml)

HTML

@model Student

```
<form asp-action="Create" method="post">

    <label>Name</label>
    <input asp-for="Name" />

    <label>Age</label>
    <input asp-for="Age" />

    <button type="submit">Submit</button>

</form>
```

## Suppose user enters

Name = Ram

Age = 21

Model Binding automatically creates:

`</>` C#

```
Student student = new Student();
```

```
student.Name = "Ram";
```

```
student.Age = 21;
```

No manual coding is required.

## Binding from Query String

URL

`https://localhost:7019/Student/Details?id=5`

Controller

`</>` C#

```
public IActionResult Details(int id)
{
    return Content("Student ID = " + id);
}
```

Output

Student ID = 5



# Binding from Route Values

Program.cs

```
</> C#  
  
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

URL

```
https://localhost:7019/Student/Edit/10
```

Controller

```
</> C#  
  
public IActionResult Edit(int id)  
{  
    return Content("Editing Student " + id);  
}
```

Output

```
Editing Student 10
```

# Model Validation

- Validation ensures that the user enters correct and complete data before it is processed or stored in the database.
- ASP.NET Core uses Data Annotations for validation.

## Common Validation Attributes

| Attribute           | Purpose                   |
|---------------------|---------------------------|
| [Required]          | Field cannot be empty     |
| [StringLength]      | Limits string length      |
| [Range]             | Limits numeric values     |
| [EmailAddress]      | Valid email format        |
| [Phone]             | Valid phone number        |
| [Compare]           | Compare two fields        |
| [RegularExpression] | Custom validation pattern |



## Example

### Model

```
</> C#  
  
using System.ComponentModel.DataAnnotations;  
  
public class Student  
{  
    [Required(ErrorMessage = "Name is required")]  
    public string Name { get; set; }  
  
    [Range(18,30)]  
    public int Age { get; set; }  
  
    [EmailAddress]  
    public string Email { get; set; }  
}
```

### View

```
@model Student
```

```
<form asp-action="Create" method="post">
```

```
    <label asp-for="Name"></label>
```

```
    <input asp-for="Name"/>
```

```
    <span asp-validation-for="Name"></span>
```

```
<br>
```

```
    <label asp-for="Age"></label>
```

```
    <input asp-for="Age"/>
```

```
    <span asp-validation-for="Age"></span>
```

```
<br>
```

```
    <label asp-for="Email"></label>
```

```
    <input asp-for="Email"/>
```

```
    <span asp-validation-for="Email"></span>
```

```
<br>
```

```
    <button type="submit">Save</button>
```

```
</form>
```

## Controller

```
</> C#  
  
[HttpPost]  
public IActionResult Create(Student student)  
{  
    if (ModelState.IsValid)  
    {  
        return Content("Data Saved Successfully");  
    }  
  
    return View(student);  
}
```

## Invalid Input

```
Name =  
Age = 15  
Email = abc
```

## Validation Messages

```
Name is required.  
  
Age must be between 18 and 30.  
  
Invalid email address.
```

### Advantages of Model Validation

- *Prevents invalid data*
- *Improves data quality*
- *Reduces coding effort*
- *Provides automatic client- and server-side validation*
- *Enhances user experience*

# URL Routing and Features

*Routing is the mechanism that maps an incoming URL to the appropriate controller and action method.*

For example:

`https://localhost:7019/Student/Index`

Here,

- **Student** → Controller
- **Index** → Action
- **7019** → Port number

Without routing, ASP.NET Core would not know which action should handle the request.

## Default Routing

In Program.cs

`</> C#`

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

### Meaning

| Part       | Description        |
|------------|--------------------|
| controller | Controller name    |
| action     | Action method      |
| id?        | Optional parameter |

## Example URLs

| URL             | Controller | Action | ID |
|-----------------|------------|--------|----|
| /               | Home       | Index  | —  |
| /Home/About     | Home       | About  | —  |
| /Student/Index  | Student    | Index  | —  |
| /Student/Edit/5 | Student    | Edit   | 5  |

### URL

https://localhost:7019/Student/Details/5

### Output

Student ID = 5

## Example Controller

```
</> C#  
  
public class StudentController : Controller  
{  
    public IActionResult Index()  
    {  
        return View();  
    }  
  
    public IActionResult Details(int id)  
    {  
        return Content("Student ID = " + id);  
    }  
}
```

# Attribute Routing

Instead of defining routes only in `Program.cs`, you can define them directly on controllers or actions using attributes.

## Example

```
</> C#  
  
[Route("students")]  
public class StudentController : Controller  
{  
    [Route("list")]  
    public IActionResult Index()  
    {  
        return View();  
    }  
}
```

## URL

`https://localhost:7019/students/list`

## Route Parameters

```
</> C#  
  
[Route("Student/Details/{id}")]  
public IActionResult Details(int id)  
{  
    return Content("ID = " + id);  
}
```

## URL

`https://localhost:7019/Student/Details/100`

## Output

ID = 100

## Named Routes

```
</> C#  
  
app.MapControllerRoute(  
    name: "StudentRoute",  
    pattern: "Student/{action}/{id?}");
```

*A named route can be referred to elsewhere in the application when generating URLs.*

# Routing Features

## 1. Default Route

Automatically routes requests using the default pattern.

```
/Home/Index
```

## 2. Attribute Routing

Routes are defined using attributes.

```
</> C#  
  
[Route("students/list")]
```

## 3. Route Parameters

Pass values through the URL.

```
/Student/Edit/5
```

## 4. Optional Parameters

The `?` symbol makes a parameter optional.

```
{id?}
```

Example

```
/Home/Index  
/Home/Index/5
```

Both URLs are valid.

## 5. URL Generation

Tag Helpers generate URLs automatically.

 HTML

```
<a asp-controller="Student"  
    asp-action="Index">  
    Students  
</a>
```

Generated HTML

 HTML

```
<a href="/Student/Index">Students</a>
```

## 6. SEO-Friendly URLs

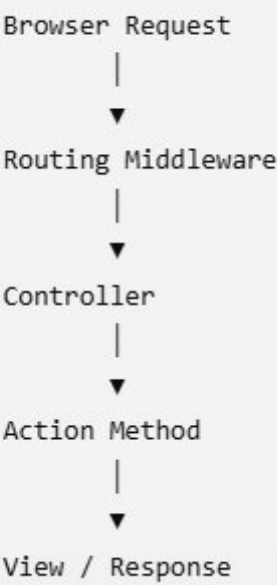
Instead of

`/Home/Details?id=10`

Routing can produce cleaner URLs:

`/Home/Details/10`

## Routing Flow



## Model Binding vs Validation

| Model Binding                                               | Validation                                      |
|-------------------------------------------------------------|-------------------------------------------------|
| Transfers request data into model objects                   | Checks whether the data is valid                |
| Executes before validation                                  | Executes after model binding                    |
| Automatically creates model objects                         | Uses Data Annotation attributes                 |
| Example: Creates <code>Student</code> object from form data | Example: Ensures <code>Name</code> is not empty |

## Conventional Routing vs Attribute Routing

| Conventional Routing                         | Attribute Routing                                       |
|----------------------------------------------|---------------------------------------------------------|
| Defined centrally in <code>Program.cs</code> | Defined using <code>[Route]</code> attributes           |
| Easy for common URL patterns                 | More flexible for custom URLs                           |
| Uses a shared route template                 | Each controller/action can have its own route           |
| Best for standard MVC applications           | Best for APIs or applications needing customized routes |



# Web API Applications

*A Web API (Application Programming Interface) is a framework that allows applications to communicate with each other over the internet using the HTTP protocol. ASP.NET Core Web API is commonly used to create RESTful services that exchange data in JSON format.*

## Example:

A mobile banking app requests account details from the bank server through a Web API.

```
Mobile App  --->  Web API  --->  Database
                |
                JSON Data
```

# API Controllers

*An **API Controller** is a class that receives HTTP requests (GET, POST, PUT, DELETE) and returns data instead of HTML pages.*

*Unlike MVC Controllers:*

- *MVC Controller → Returns Views (HTML)*
- *API Controller → Returns Data (JSON/XML)*

**API controllers inherit from ControllerBase instead of Controller**

They are usually decorated with:

```
</> C#  
  
[ApiController]  
[Route("api/[controller]")]
```

## Example: Student API Controller

```
</> C#  
  
using Microsoft.AspNetCore.Mvc;  
  
namespace DemoAPI.Controllers  
{  
    [ApiController]  
    [Route("api/[controller]")]  
    public class StudentController : ControllerBase  
    {  
        [HttpGet]  
        public IActionResult GetStudents()  
        {  
            var students = new[]  
            {  
                new { Id = 1, Name = "Ram", Age = 20 },  
                new { Id = 2, Name = "Sita", Age = 21 }  
            };  
  
            return Ok(students);  
        }  
    }  
}
```

## URL

```
https://localhost:5001/api/student
```

## Output (JSON)

</> JSON

```
[
  {
    "id":1,
    "name":"Ram",
    "age":20
  },
  {
    "id":2,
    "name":"Sita",
    "age":21
  }
]
```

## Common HTTP Methods

| Method | Purpose       | Example                |
|--------|---------------|------------------------|
| GET    | Retrieve data | Get all students       |
| POST   | Insert data   | Add new student        |
| PUT    | Update data   | Update student details |
| DELETE | Delete data   | Delete student         |

## Example of POST

</> C#

```
[HttpPost]
public IActionResult AddStudent(Student student)
{
    return Ok(student);
}
```

# JSON (JavaScript Object Notation)

- *JSON is a lightweight text format used to exchange data between client and server.*
- *Almost every ASP.NET Core Web API sends and receives data in JSON.*

## Example Student Object

JSON

```
{  
  "id":1,  
  "name":"Ram",  
  "age":20  
}
```

## Why JSON?

- Lightweight
- Easy to read
- Language independent
- Faster than XML
- Supported by almost every programming language

## C# Object

</> C#

```
Student s = new Student
{
    Id = 1,
    Name = "Ram",
    Age = 20
};
```

Automatically converted into JSON

</> JSON

```
{
  "id":1,
  "name":"Ram",
  "age":20
}
```

## Receiving JSON

Client sends

</> JSON

```
{
  "name":"Hari",
  "age":22
}
```

*ASP.NET Core automatically converts  
JSON into a C# object (**Model Binding**).*

Controller

</> C#

```
[HttpPost]
public IActionResult Add(Student student)
{
    return Ok(student);
}
```

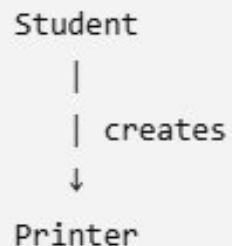
# Dependency Injection (DI)

*Dependency Injection (DI) is a design technique in which an object receives the other objects it depends on from outside, instead of creating those objects itself.*

***Don't create the dependency yourself; ask someone else to provide it.***

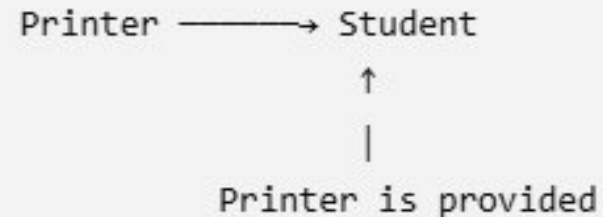
Suppose a **Student** needs a **Printer**.

Without Dependency Injection:



The Student class is responsible for creating the Printer.

With Dependency Injection:



The Student simply says:

"I need a Printer."

Something else provides the Printer.

## Without Dependency Injection

Suppose we have a `Student` class:

```
</> C#  
  
public class Student  
{  
    public void PrintResult()  
    {  
        Printer printer = new Printer();  
        printer.Print();  
    }  
}
```

And:

```
</> C#  
  
public class Printer  
{  
    public void Print()  
    {  
        Console.WriteLine("Printing student result...");  
    }  
}
```

Here, `Student` directly creates the `Printer`:

```
</> C#  
  
Printer printer = new Printer();
```

This creates **tight coupling** between `Student` and `Printer`.

If later we want to replace `Printer` with `PDFPrinter`, we have to modify the `Student` class.



# With Dependency Injection

First, create an interface:

```
</> C#  
  
public interface IPrinter  
{  
    void Print();  
}
```

Create a printer:

```
</> C#  
  
public class Printer : IPrinter  
{  
    public void Print()  
    {  
        Console.WriteLine("Printing student result...");  
    }  
}
```

Now modify the `Student` class:

```
</> C#  
  
public class Student  
{  
    private readonly IPrinter _printer;  
  
    public Student(IPrinter printer)  
    {  
        _printer = printer;  
    }  
  
    public void PrintResult()  
    {  
        _printer.Print();  
    }  
}
```

Notice this:

```
</> C#  
  
public Student(IPrinter printer)
```

The `Student` class **does not create a Printer**.

Instead, a printer is **injected into the constructor**.

This is called **Constructor Dependency Injection**.

*Dependency Injection (DI) is a design pattern in which the required objects (dependencies) are provided to a class instead of the class creating them itself.*

#### Without DI

```
</> C#  
  
public class StudentController  
{  
    StudentService service = new StudentService();  
}
```

#### Problems:

- Tight coupling
- Hard to test
- Difficult to maintain

#### With DI

```
</> C#  
  
public class StudentController  
{  
    private readonly StudentService _service;  
  
    public StudentController(StudentService service)  
    {  
        _service = service;  
    }  
}
```

*The framework automatically creates the StudentService object and injects it into the controller.*

## 4. IoC (Inversion of Control) Container

Inversion of Control (IoC) is a design principle where the creation and management of objects is handled by a framework or container instead of the application code.

The IoC Container in ASP.NET Core automatically:

- *Creates objects*
- *Manages object lifetimes*
- *Resolves dependencies*
- *Injects required services*

***IoC means transferring the responsibility/control of creating and managing objects from the application classes to another system.***

**IoC** stands for **Inversion of Control**.

Normally, a class controls the creation of its dependencies:

```
Student
  ↓
creates Printer
```

With IoC, the control is reversed:

```
IoC Container
  ↓
creates Printer
  ↓
provides Printer to Student
```

***Dependency Injection is one of the most common ways of implementing IoC.***

## Example

### Service Registration

</> C#

```
builder.Services.AddScoped<StudentService>();
```

### Controller

</> C#

```
public StudentController(StudentService service)
{
    _service = service;
}
```

The IoC container:

1. Sees that `StudentController` needs a `StudentService`.
2. Creates a `StudentService` object.
3. Passes it to the controller constructor.

No manual object creation is needed.

## Difference Between DI and IoC

### Dependency Injection (DI)

*Technique for supplying dependencies to a class*

*Makes classes loosely coupled*

*Implemented using constructor, property, or method injection*

### Inversion of Control (IoC)

*Design principle where object creation is controlled externally*

*Achieved using an IoC container*

*IoC container manages object creation and dependency resolution*

# ASP.NET Core Example on DI and IoC

Suppose we have:

## Step 1: Interface

```
</> C#  
  
public interface IMessageService  
{  
    string GetMessage();  
}
```

## Step 2: Implementation

```
</> C#  
  
public class MessageService : IMessageService  
{  
    public string GetMessage()  
    {  
        return "Hello from Message Service!";  
    }  
}
```

## Step 3: Register the dependency

In `Program.cs` :

```
</> C#  
  
var builder = WebApplication.CreateBuilder(args);  
  
builder.Services.AddScoped<IMessageService, MessageService>();  
  
builder.Services.AddControllers();  
  
var app = builder.Build();  
  
app.MapControllers();  
  
app.Run();
```

The important line is:

```
</> C#  
  
builder.Services.AddScoped<IMessageService, MessageService>();
```

It registers the dependency with the **IoC container**.

## Step 4: Inject it into a Controller

</> C#

```
public class StudentController : Controller
{
    private readonly IMessageService _messageService;

    public StudentController(IMessageService messageService)
    {
        _messageService = messageService;
    }

    public IActionResult Index()
    {
        string message = _messageService.GetMessage();

        return Content(message);
    }
}
```

We never write:

</> C#

```
new MessageService()
```

inside the controller.

Instead, ASP.NET Core automatically creates `MessageService` and passes it to:

</> C#

```
StudentController(IMessageService messageService)
```



# DI Service Lifetimes in ASP.NET Core

| Lifetime  | Meaning                                                                 | Simple Example                                     |
|-----------|-------------------------------------------------------------------------|----------------------------------------------------|
| Transient | A new object every time it is requested.                                | AddTransient<IEmailService, EmailService>()        |
| Scoped    | One object per HTTP request. Same object is reused within that request. | AddScoped<IUserService, UserService>()             |
| Singleton | One object for the entire application. Reused by all requests.          | AddSingleton<IConfigService, ConfigService><br>( ) |

1. Transient – New instance every time requested.

```
</> C#  
  
builder.Services.AddTransient<IMyService, MyService>();
```

2. Scoped – One instance per HTTP request.

```
</> C#  
  
builder.Services.AddScoped<IMyService, MyService>();
```

3. Singleton – One instance for the entire application.

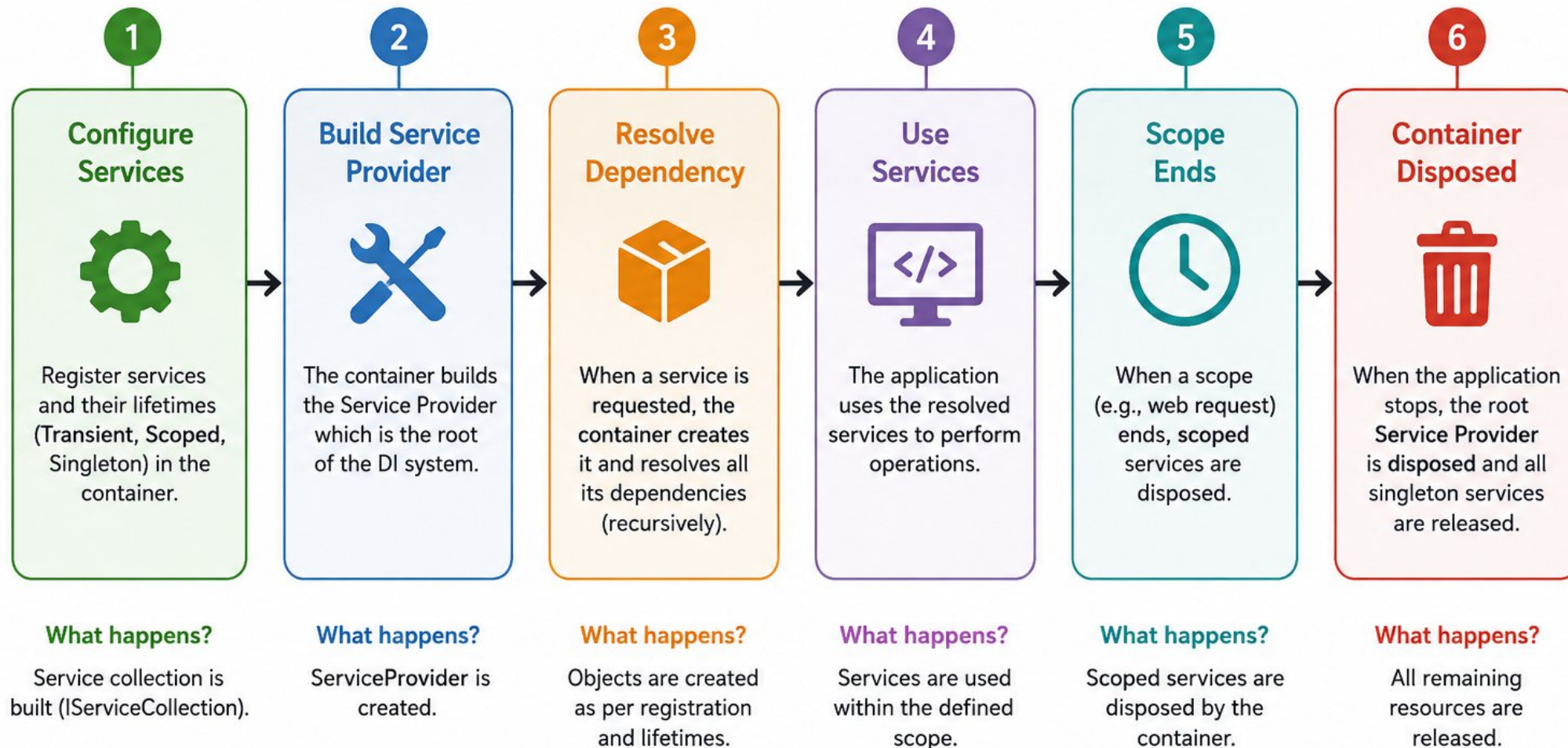
```
</> C#  
  
builder.Services.AddSingleton<IMyService, MyService>();
```

- Transient → New every time 
- Scoped → One per request 
- Singleton → One for application 



# Life Cycle of Dependency Injection (DI) Container

The DI container is responsible for managing the creation, resolution, and disposal of objects and their dependencies.



**Register → Build → Resolve → Use → Scope Ends → Dispose**

This is the complete life cycle of a DI container.

## 1. Create the Model

Create `Models/User.cs` :

```
</> C#  
  
using System.ComponentModel.DataAnnotations;  
  
namespace MyMvcApp.Models  
{  
    public class User  
    {  
        [Required]  
        public string Name { get; set; }  
  
        [Required]  
        [EmailAddress]  
        public string Email { get; set; }  
  
        [Required]  
        public string Phone { get; set; }  
  
        [Range(18, 100)]  
        public int Age { get; set; }  
    }  
}
```

*ASP.NET Core MVC example that collects user info (Name, Email, Phone, Age) and displays it with basic validations*

### Validation used

| Attribute        | Purpose                        |
|------------------|--------------------------------|
| [Required]       | Field cannot be empty          |
| [EmailAddress]   | Checks valid email format      |
| [Range(18, 100)] | Age must be between 18 and 100 |

## 2. Create the Controller

Create `Controllers/UserController.cs` :

</> C#

```
using Microsoft.AspNetCore.Mvc;
using MyMvcApp.Models;

namespace MyMvcApp.Controllers
{
    public class UserController : Controller
    {
        // Display form
        public IActionResult Index()
        {
            return View();
        }

        // Receive form data
        [HttpPost]
        public IActionResult Index(User user)
        {
            if (ModelState.IsValid)
            {
                return View("Result", user);
            }

            return View(user);
        }
    }
}
```

### How it works

- `Index()` → displays the form.
- `[HttpPost] Index(User user)` → receives the submitted information.
- `ModelState.IsValid` → checks whether validation is successful.
- If valid → displays the `Result` page.
- If invalid → returns to the form and displays errors.

### 3. Create the View

Create:

Views/User/Index.cshtml

#### HTML

```
<input asp-for="Email" />
<span asp-validation-for="Email"></span>
<br /><br />

<label>Phone:</label>
<input asp-for="Phone" />
<span asp-validation-for="Phone"></span>
<br /><br />

<label>Age:</label>
<input asp-for="Age" />
<span asp-validation-for="Age"></span>
<br /><br />

<button type="submit">Submit</button>

</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

### 4. Create the Result View

Create:

Views/User/Result.cshtml

#### HTML

```
@model MyMvcApp.Models.User

<h2>User Information</h2>

<p><b>Name:</b> @Model.Name</p>
<p><b>Email:</b> @Model.Email</p>
<p><b>Phone:</b> @Model.Phone</p>
<p><b>Age:</b> @Model.Age</p>

<a asp-action="Index">Back to Form</a>
```

### 5. Run the Application

Run the project and open:

https://localhost:xxxx/User



How do you create controller? Illustrate with an example. Describe the procedure of rendering HTML with Razor with scenario.

Describe the life cycle of Dependency Injection container.

What are different Action Result Types in ASP.NET Core MVC? Explain with examples. Design an API Controller that returns JSON response for a Product entity (ProductId, Name, Price, Category) with GET, POST, PUT, and DELETE operations.

How do you create controller? Mention some requirements for rendering HTML.

Describe the importance of MVC pattern in designing web applications.

Mention the two types of services managed by IoC container and how they are registered?

What are tag helpers? What tag helper provides?

What does dependency injection and IOC container means? Explain.

What is tag helper? Discuss about razor syntax with an example.

Give an example of validation in MVC framework.